Prácticas de seguridad de MongoDB 8.0

Laboratorio 1 — Un lector puede consultar, pero no escribir

Objetivo: comprobar tres situaciones reales: lectura anónima rechazada; lectura autorizada al usuario `lectorlab`; intento de inserción rechazado para el mismo usuario.

Paso 1 — Saber qué servidor utilizaremos

Este laboratorio usa `mongo-seguridad`, **no** `mongo-indices`. Se trata de dos instancias de MongoDB diferentes dentro del mismo proyecto. La base de datos se llama `curso_seguridad` y su colección de práctica, `secretos`.

Abre PowerShell o Terminal en la raíz del repositorio (debe existir `compose.yaml`). Si no has preparado `.env`, copia `.env.example` y cambia las contraseñas de demostración `MONGO_ADMIN_PASSWORD`, `LAB_READER_PASSWORD` y `LAB_WRITER_PASSWORD`. No uses contraseñas personales.

PowerShell:

```
Copy-Item .env.example .env
```

Bash:

```
cp .env.example .env
```

Paso 2 — Arrancar la instancia protegida

En PowerShell o Bash, no dentro de `mongosh`:

```
docker compose up -d --wait mongo-seguridad
docker compose ps
```

Comprueba que `mongo-seguridad` aparece en ejecución. Si cambias la contraseña en `.env` después de haber creado la instancia con otros datos persistentes, eso no actualiza automáticamente al usuario existente. Consulta con el formador antes de eliminar volúmenes.

Paso 3 — Entrar como administrador

```
docker compose exec mongo-seguridad mongosh --username adminlab --authenticationDatabase admin
--password
```

Introduce la clave de `MONGO_ADMIN_PASSWORD` cuando se solicite. Ahora estás dentro de `mongosh`. Confirma la conexión:

```
db.adminCommand({ping:1})
```

Debe aparecer `ok:1`. Esto demuestra que el servidor responde, **no** que todas las cuentas ya existan.

Paso 4 — Preparar un documento ficticio

Todavía dentro de `mongosh`:

```
const curso = db.getSiblingDB("curso_seguridad")
curso.secretos.insertOne({
  _id: "demo-1",
  texto: "registro autorizado",
  clasificacion: "docente"
})
```

`getSiblingDB` selecciona otra base de datos sin cerrar tu conexión. `insertOne` crea un documento. Si aparece error de identificador duplicado, quizás realizaste antes la práctica: comprueba `curso.secretos.findOne({_id:"demo-1"})` y evita repetir la inserción.

Paso 5 — Crear los usuarios con permisos diferentes

Con la conexión administrativa:

```
curso.createUser({
  user:"lectorlab",
  pwd:process.env.LAB_READER_PASSWORD,
  roles:[{role:"read",db:"curso_seguridad"}]
})
```

El ejemplo obtiene la contraseña reservada al lector desde las variables del contenedor que Compose carga de `.env`, y evita escribir la clave en la guía. Después:

```
curso.createUser({
  user:"escritorlab",
  pwd:process.env.LAB_WRITER_PASSWORD,
  roles:[{role:"readWrite",db:"curso_seguridad"}]
})
```

Importante: si los usuarios ya existen, MongoDB lo indicará. No los borres automáticamente; consulta al formador o comprueba su existencia con `curso.getUser("lectorlab")`.

`read` permite consultar documentos; `readWrite` permite además modificarlos en esa base de datos.

Paso 6 — Intentar leer sin credenciales

Escribe `exit` para abandonar `mongosh`. De nuevo en PowerShell o Bash:

```
docker compose exec mongo-seguridad mongosh
```

Dentro del nuevo `mongosh` anónimo:

```
db.getSiblingDB("curso_seguridad").secretos.findOne({_id:"demo-1"})
```

Resultado esperado: error de autorización. El hecho de poder establecer una conexión no equivale a tener permiso para leer. Anota el mensaje y explica qué control ha actuado. Después escribe `exit`.

Paso 7 — Leer como usuario autorizado

Desde PowerShell o Bash:

```
docker compose exec mongo-seguridad mongosh --username lectorlab --authenticationDatabase
curso_seguridad --password
```

Escribe la contraseña del lector. Dentro de `mongosh`:

```
db.getSiblingDB("curso_seguridad").secretos.findOne({_id:"demo-1"})
```

Resultado esperado: aparece el documento con el texto `registro autorizado`. Esta vez se han cumplido dos condiciones: la identidad del lector es válida y su rol permite leer.

Paso 8 — Provocar la denegación correcta

Sin salir de la sesión del lector, ejecuta:

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({
  _id:"prohibido",
  texto:"el lector no deberia poder crear esto"
})
```

Resultado esperado: error `Unauthorized` o equivalente. Esta es precisamente la protección que queríamos demostrar.

Si la inserción funciona, has encontrado un problema en la preparación: comprueba qué usuario está conectado y los roles que le fueron asignados. No continúes fingiendo que el ejercicio ha pasado.

Paso 9 — Contrastar con el usuario escritor

Sal con `exit`. Conecta con la cuenta que sí debe escribir:

```
docker compose exec mongo-seguridad mongosh --username escritorlab --authenticationDatabase
curso_seguridad --password
```

Dentro de `mongosh`:

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({
  _id:"permitido",
  texto:"el escritor si puede insertar"
})
```

Resultado esperado: inserción confirmada.

Paso 10 — Registrar la evidencia y cerrar

Prueba	Resultado correcto	Resultado observado
Anónimo intenta leer	Denegado	____
Lector intenta leer	Permitido	____
Lector intenta insertar	Denegado	____
Escritor intenta insertar	Permitido	____

Explica por escrito por qué **no** sería correcto resolver la tercera prueba dando al lector permisos administrativos.

Sal de `mongosh` y detén el servicio desde la terminal:

```
docker compose stop mongo-seguridad
```

Este comando detiene el contenedor y conserva sus datos. La nueva configuración del proyecto utiliza **volúmenes Docker con nombre** para almacenar los datos de cada instancia; `docker compose down` elimina contenedores, pero no esos volúmenes con nombre. **`docker compose down --volumes` sí los elimina** y perderás los datos, usuarios y documentos de práctica. No lo ejecutes salvo que quieras reinicializar conscientemente el laboratorio.

Siguiente práctica: cifrado TLS y certificados X.509, comenzando por la diferencia entre canal seguro e identidad del cliente.

Laboratorio 2 — Distinguir errores de conexión, autenticación y autorización

Objetivo

Aprenderás a diagnosticar tres fallos que a veces producen mensajes parecidos, pero que tienen soluciones completamente diferentes. Esta práctica no consiste en «hacer que todo funcione»: algunas operaciones están diseñadas para ser rechazadas. Debes explicar **cuál es la condición que falla**.

Continuamos con los usuarios y datos preparados en la primera práctica. Si todavía no has creado `lectorlab` y `escritorlab`, realiza el laboratorio 1 antes de seguir.

Paso 1. Distinguir los tres planos

Imagina que te presentas ante una puerta:

- Si el edificio está cerrado y no existe acceso físico, no puedes siquiera contactar con el responsable de recepción.
- Si consigues llegar pero te identificas con una credencial incorrecta, **falla la autenticación**.
- Si te identificas correctamente pero intentas entrar en un área restringida, **falla la autorización**.

El laboratorio reproduce las tres situaciones utilizando un servicio, unas credenciales y una operación.

Paso 2. Comprobar la disponibilidad del servicio

Desde PowerShell o Bash en la raíz del proyecto:

```
docker compose ps
```

Comprueba que `mongo-seguridad` está en marcha. Si está detenido, arráncalo:

```
docker compose up -d --wait mongo-seguridad
```

¿Aparece como `healthy`? Si no, todavía no intentes interpretar errores de contraseñas. Un servidor que no arrancó no puede validar credenciales.

Paso 3. Provocar una denegación por no autenticarse

En una terminal abre `mongosh` sin nombre ni contraseña:

```
docker compose exec mongo-seguridad mongosh
```

Dentro del shell:

```
db.getSiblingDB("curso_seguridad").secretos.findOne({_id:"demo-1"})
```

Debe obtenerse una denegación de acceso. El error significa que el servidor está activo y ha recibido la solicitud, pero no ha autorizado leer esa colección.

No concluyas que el puerto está bloqueado: de hecho, llegaste al servidor. Sal mediante `exit`.

Paso 4. Probar una contraseña equivocada

Conecta desde la terminal como `lectorlab`, pero introduce voluntariamente una contraseña diferente de la que preparaste:

```
docker compose exec mongo-seguridad mongosh --username lectorlab --authenticationDatabase
curso_seguridad --password
```

Aquí se espera un fallo de **autenticación**. La cuenta no ha demostrado conocer el secreto correcto, por lo que no puede ser reconocida como `lectorlab`.

Registra el texto del error o la información esencial, **sin escribir contraseñas en tu informe**. Si la consola muestra de nuevo el prompt de autenticación, no repitas intentos indefinidamente.

Paso 5. Probar la base de autenticación incorrecta

Ahora prueba a iniciar sesión con las credenciales correctas del lector pero indicando `admin`:

```
docker compose exec mongo-seguridad mongosh --username lectorlab --authenticationDatabase admin
--password
```

En este laboratorio la cuenta `lectorlab` se creó asociada a `curso_seguridad`, no a `admin`. Por tanto, esperamos un fallo de autenticación en esa combinación.

Pregunta: **¿qué cambió respecto a un inicio de sesión correcto?** No fue necesariamente el usuario ni la contraseña: cambió el lugar donde el servidor busca esa identidad.

Paso 6. Conectar correctamente y probar lectura

```
docker compose exec mongo-seguridad mongosh --username lectorlab --authenticationDatabase
curso_seguridad --password
```

Introduce ahora la contraseña verdadera del lector. Dentro de `mongosh` ejecuta:

```
db.getSiblingDB("curso_seguridad").secretos.findOne({_id:"demo-1"})
```

Debe aparecer el documento de prueba. Esta comprobación es imprescindible: sin una prueba de lectura permitida no sabemos si la cuenta está preparada de forma correcta.

Paso 7. Probar dos escrituras diferentes

Todavía conectado como lector, intenta modificar el documento:

```
db.getSiblingDB("curso_seguridad").secretos.updateOne(
  {_id:"demo-1"},
  {$set:{texto:"modificado sin permiso"}}
)
```

Resultado esperado: denegación. El filtro identifica un documento existente, pero el rol `read` no permite actualizarlo.

Después intenta eliminarlo:

```
db.getSiblingDB("curso_seguridad").secretos.deleteOne({_id:"demo-1"})
```

Resultado esperado: denegación otra vez. El mismo rol impide la eliminación.

Observa la diferencia respecto al paso 4. **Aquí no falla la autenticación:** has podido leer con esta misma sesión. Lo que falla es que intentas una acción no autorizada.

Paso 8. Comprobar que nada se modificó

```
db.getSiblingDB("curso_seguridad").secretos.findOne({_id:"demo-1"})
```

El campo `texto` debería conservar su valor original `registro autorizado`. Así comprobamos que las denegaciones protegieron los datos, no simplemente que apareció un mensaje de error.

Si el documento ha cambiado, es posible que estuvieras conectado con una identidad más privilegiada. Detente, registra el problema y consulta qué usuario está realmente autenticado antes de repetir la prueba.

Paso 9. Completar un diagnóstico por causas

Experimento	El servidor respondió	Credenciales válidas	Permiso para la operación
Lectura anónima	Sí	No	No
Contraseña equivocada	Sí	No	No se llega a comprobar
Base de autenticación incorrecta	Sí	No	No se llega a comprobar
Lectura como lector	Sí	Sí	Sí
Actualizar como lector	Sí	Sí	No
Borrar como lector	Sí	Sí	No

Esta tabla no es un ejercicio de memoria: explica cada fila con el comando que ejecutaste y el resultado observado.

Paso 10. Resolver errores de preparación

Si la contraseña correcta no funciona, verifica:

1. que el usuario fue creado en el laboratorio 1;
2. que lo estás buscando en `curso_seguridad`;
3. que `.env` contiene la contraseña que utilizaste originalmente;
4. que no reutilizaste una instancia de MongoDB inicializada con otra configuración.

No borres volúmenes para «arreglar la autenticación» sin consultar primero al formador: perderías cuentas y documentos de la práctica.

Laboratorio 3 — TLS: probar un canal seguro, no solo una contraseña

Qué vamos a conseguir

La instancia `mongo-tls` exige TLS. Vamos a demostrar dos resultados complementarios:

1. una conexión **sin TLS** es rechazada;
2. una conexión TLS **que valida la CA local** puede hablar con el servidor.

Ninguna de esas dos pruebas significa por sí sola que el cliente tenga permisos para consultar las colecciones protegidas. El objetivo es distinguir **protección del canal** de **autorización de datos**.

Paso 1. Requisitos concretos

Este laboratorio requiere:

- Docker Desktop o Docker Engine con Compose;
- Bash y OpenSSL, por ejemplo mediante Git Bash o WSL en Windows;
- la carpeta del proyecto con `compose.yaml` y el script `scripts/generar-certificados.sh`;
- un archivo `.env` con una contraseña de demostración para `MONGO_ADMIN_PASSWORD`.

Nunca utilices certificados de producción ni contraseñas personales. El procedimiento genera una autoridad de certificación de laboratorio y material de clave con permisos simplificados para contenedores desechables.

Paso 2. Preparar los certificados locales

Abre **Bash** en la carpeta principal del repositorio:

```
bash scripts/generar-certificados.sh
```

Debes observar una confirmación de verificación criptográfica. En la carpeta `certs` aparecerán, entre otros, `ca.crt`, `server.pem` y `client.pem`.

Qué representa cada archivo:

- `ca.crt`: certificado de la CA con el que se verifica la cadena de confianza.
- `server.pem`: certificado de servidor y material asociado necesario para presentarlo.
- `client.pem`: material de certificado del alumno para el laboratorio X.509.

No compartas `server.pem` ni `client.pem`: en este laboratorio incluyen claves privadas. La carpeta `certs` está excluida del control de versiones.

Paso 3. Arrancar el servidor que exige TLS

```
docker compose up -d --wait mongo-tls
docker compose ps
```

Debe aparecer `mongo-tls` en ejecución. El puerto del host es `127.0.0.1:27019`, pero las órdenes del laboratorio se ejecutan dentro del contenedor y usan `localhost`, donde MongoDB escucha en su puerto interno.

Si no arranca, consulta:

```
docker compose logs --tail=100 mongo-tls
```

Errores relacionados con certificado, clave o archivo inexistente requieren volver a comprobar el paso 2.

Paso 4. Intentar conectar SIN TLS

Desde PowerShell o Bash:

```
docker compose exec -T mongo-tls mongosh --quiet --host localhost --eval
'db.adminCommand({ping:1})'
```

Resultado esperado: la conexión falla. No lo registres como fracaso del ejercicio: el servidor está configurado para exigir un canal cifrado.

La pregunta que debes responder es: ¿qué intentaste hacer y qué propiedad de seguridad impidió que se aceptara?

Paso 5. Repetir la conexión con TLS y CA

```
docker compose exec -T mongo-tls mongosh --quiet --host localhost --tls --tlsCAFile /certs/
ca.crt --eval 'printjson(db.adminCommand({ping:1}))'
```

Ahora introduces dos elementos:

- `--tls`: pide un canal TLS.
- `--tlsCAFile /certs/ca.crt`: proporciona la CA de confianza que permitirá verificar al servidor.

Resultado esperado: documento con `ok:1`.

Muy importante: `ping` confirma que el servidor es accesible usando ese canal; no demuestra todavía que esa conexión pueda leer una colección protegida.

Paso 6. Explicar qué cambió y qué no cambió

Completa:

Prueba	TLS utilizado	Resultado esperado
`ping` sin `--tls`	No	Conexión rechazada
`ping` con `--tls` y CA	Sí	Respuesta `ok:1`

El servidor no empezó a permitir escritura al activar TLS. Lo que cambió fue que el cliente negoció un canal protegido. Las decisiones sobre identidad y roles se estudiarán en el laboratorio siguiente.

Paso 7. Ver el certificado sin exponer su clave

En Bash, desde la raíz del proyecto:

```
openssl x509 -in certs/server.crt -noout -subject -issuer -dates
```

Observa:

- `subject`: la identidad para la que se emitió el certificado;
- `issuer`: su emisor;
- `notBefore` y `notAfter`: su intervalo de validez.

No pegues claves privadas ni contraseñas en el informe. Los metadatos del certificado bastan para esta actividad.

Paso 8. Preguntas antes de continuar

1. Si `ping` funciona con TLS, ¿significa que puedes leer todos los datos?
2. ¿Qué función cumple la CA?
3. ¿Cuál de los archivos debemos proteger especialmente porque incorpora material de clave privada?
4. ¿Por qué es incorrecto solucionar un error de confianza desactivando indiscriminadamente la validación de certificados?

Laboratorio 4 — X.509 y privilegios: autenticarse con un certificado de cliente

4.1. La pregunta que cambia

En el laboratorio anterior el cliente verificó al servidor. Ahora queremos que **el servidor identifique al cliente** mediante un certificado X.509.

El mecanismo de MongoDB que vamos a utilizar se llama `MONGODB-X509`. La identidad vive en la base especial `$external`, pero sus privilegios se asignan igualmente mediante roles.

Vamos a demostrar:

- el cliente puede autenticarse mediante su certificado;
- con un rol de lectura puede consultar un documento;
- la misma identidad no puede insertar.

Paso 1. Trabajar sobre un entorno de laboratorio nuevo

El script de comprobación completo crea cuentas de ejemplo, incluida la cuenta administrativa inicial de la instancia TLS. Se diseñó para ejecutarse sobre contenedores **desechables recién creados**. Si ya has creado manualmente `admincert` o las cuentas de ejemplo, volver a ejecutar todas las operaciones de inicialización puede fallar por usuarios existentes.

Antes de repetir el flujo completo, pregunta al formador si la instancia `mongo-tls` contiene algo que deba conservarse. No reinicies ni elimines datos reales.

La validación automatizada de GitHub Actions utiliza un entorno limpio en cada ejecución precisamente para evitar ambigüedades de este tipo.

Paso 2. Preparar un shell Bash y las variables

Desde la raíz del repositorio, comprueba que has generado certificados con el script anterior. Utiliza una copia local de `.env` creada y revisada por ti; no ejecutes ni importes archivos de configuración de origen desconocido.

Para pasar las contraseñas de demostración al entorno de **ese Bash**, puedes usar:

```
set -a
source ./.env
set +a
```

Esto carga valores definidos en `.env`. **Solo hazlo con un archivo de tu propia carpeta que hayas inspeccionado**, porque `source` ejecuta sintaxis de shell. Si utilizas una configuración de claves más avanzada, exporta las variables según las instrucciones de tu plataforma en lugar de ejecutar archivos ajenos.

Las tres variables que necesita el script son `MONGO_ADMIN_PASSWORD`, `LAB_READER_PASSWORD` y `LAB_WRITER_PASSWORD`.

Paso 3. Ejecutar el flujo de validación completo

Con los servicios de seguridad preparados y el entorno limpio:

```
docker compose up -d --wait mongo-seguridad mongo-tls
bash scripts/probar-seguridad.sh
```

El script lleva a cabo varias verificaciones, no solamente X.509. Entre otras cosas:

1. Prepara las cuentas SCRAM de `mongo-seguridad`.
2. Comprueba rechazos esperados por ausencia o error de credenciales.
3. Comprueba permisos de lectura y escritura.
4. Confirma que `mongo-tls` exige TLS.
5. Registra el usuario de certificado y prueba su acceso.
6. Exige una **denegación** cuando el usuario X.509 de solo lectura intenta insertar.

No des por aprobado el ejercicio si el script termina antes del mensaje final de éxito. Si aparece error, identifica en qué paso se produjo. La prueba está automatizada para poder reproducirla de la misma forma que los controles ejecutados en GitHub.

Paso 4. Entender la identidad del certificado

Puedes inspeccionar el sujeto del certificado del alumno sin mostrar su clave privada:

```
openssl x509 -in certs/client.crt -noout -subject -nameopt RFC2253
```

Verás un DN que representa una identidad como `CN=alumno-x509,OU=Alumnos,O=MongoDBFormacion`.

En el script `lab/tls-usuario-x509.js`, la identidad se registra en `$external` con un rol `read` sobre `curso_cert`. No es necesario copiar contraseñas dentro de ese usuario: la autenticación se basa en su certificado y clave, con la configuración correspondiente del servidor.

Paso 5. Reconocer las opciones del cliente

El flujo de validación utiliza una combinación conceptualmente equivalente a:

```
mongosh --tls --tlsCAFile /certs/ca.crt --tlsCertificateKeyFile /certs/client.pem --
authenticationMechanism MONGODB-X509 --authenticationDatabase '$external'
```

Este fragmento muestra **las opciones esenciales**: en una conexión real también debes indicar el servidor adecuado y la ruta de los archivos, que aquí son rutas internas del contenedor. La ejecución completa, con argumentos correctos, está incluida en `scripts/probar-seguridad.sh`.

- `--tls` establece el canal protegido.
- `--tlsCAFile` valida la CA del servidor.
- `--tlsCertificateKeyFile` presenta las credenciales del cliente.
- `--authenticationMechanism MONGODB-X509` indica el tipo de autenticación.
- `--authenticationDatabase '$external'` indica el almacén especial de identidades externas.

Observa las comillas alrededor de `'$external'`: impiden que Bash interprete ese símbolo como una variable de shell.

Paso 6. Distinguir las dos pruebas finales

El script intenta leer un documento protegido en `curso_cert.pruebas` con el certificado del alumno. Eso debe funcionar si la identidad y su rol de lectura están configurados.

Después intenta insertar otro documento. Eso debe fallar porque la cuenta únicamente posee lectura.

Conclusión correcta: X.509 logró demostrar la identidad; RBAC limitó sus acciones. No hay contradicción entre «usuario autenticado» y «escritura rechazada».

Paso 7. Resolver fallos con criterio

- **El servidor no arranca:** revisa certificado de servidor, rutas y permisos de archivos.
- **Falla la conexión TLS:** comprueba la CA, el certificado y el nombre de servidor.
- **TLS funciona, pero X.509 no autentica:** inspecciona el DN, la cuenta registrada en `$external` y el mecanismo indicado.
- **X.509 autentica, pero no lee:** revisa el rol concedido y la base de datos sobre la que aplica.
- **El lector consigue insertar:** sería una anomalía de autorización: examina sus roles y comprueba que no estás conectado como administrador.

No resuelvas todos estos fallos dando permisos `root`. Cada uno pertenece a una capa diferente.

Paso 8. Síntesis escrita

Redacta cuatro frases:

1. Qué demuestra el certificado de servidor.
2. Qué demuestra el certificado de cliente.
3. Qué permite un rol `read`.
4. Qué resultado negativo debe obtener el usuario de certificado si intenta insertar.

Ampliación conceptual: distingue la función de LDAP, el cifrado de datos en reposo y las copias de seguridad. No tenemos un entorno Enterprise dentro de este laboratorio; por tanto, esos mecanismos no se presentarán como probados.

Laboratorio 5 — Crear un rol propio: una cuenta solo puede leer una colección

La necesidad real

Hasta ahora hemos utilizado el rol predefinido `read`. Con él, `lectorlab` puede consultar datos de su base según los privilegios que otorga ese rol. Pero imaginemos un requisito más restrictivo:

> «El equipo de control solo puede consultar la colección `secretos`, no cualquier otra colección de la base `curso_seguridad`».

Esta diferencia es importante. El permiso de lectura sobre **toda una base** no es idéntico al permiso sobre **una colección concreta**.

Vamos a crear un rol personalizado de lectura para una sola colección y una cuenta distinta que lo utilice. No modificaremos los roles de `lectorlab` o `escritorlab` para que las prácticas anteriores sigan siendo repetibles.

Paso 1. Preparar el entorno protegido

Abre PowerShell o Bash en la raíz del proyecto:

```
docker compose up -d --wait mongo-seguridad
```

Accede como administrador:

```
docker compose exec mongo-seguridad mongosh --username adminlab --authenticationDatabase admin
--password
```

Introduce la contraseña administrativa de tu archivo local `.env`. Dentro de `mongosh` confirma que estás conectado:

```
db.adminCommand({ping:1})
```

La respuesta `ok:1` confirma comunicación con el servidor; no sustituye las pruebas de permisos posteriores.

Paso 2. Preparar dos colecciones claramente distintas

Dentro de `mongosh`:

```
const s = db.getSiblingDB("curso_seguridad");
s.secretos.findOne({_id:"demo-1"});
s.publicaciones_demo.updateOne(
  {_id:"publica-1"},
  {$set:{titulo:"Información de ensayo"}},
  {upsert:true}
);
```

El `upsert` insertará el documento si todavía no existe o lo actualizará si existe. Estamos utilizando únicamente datos ficticios de laboratorio.

Tenemos dos colecciones:

- `secretos`: la que permitiremos consultar.
- `publicaciones_demo`: la que el nuevo usuario **no debería** poder leer.

Paso 3. Definir un rol personalizado

```
s.createRole({
  role:"soloSecretosClase",
  privileges:[
    {
      resource:{db:"curso_seguridad",collection:"secretos"},
      actions:["find"]
    }
  ],
  roles:[]
});
```

Descompongámoslo:

- `role` asigna un nombre a la definición.
- `privileges` contiene permisos explícitos.
- `resource` indica la base y colección permitidas.
- `actions:["find"]` limita la acción a consulta de documentos.
- `roles:[]` no incorpora otros roles heredados.

No hemos concedido `insert`, `update`, `remove` ni privilegios sobre `publicaciones_demo`.

Si el comando informa de que el rol ya existe, seguramente repetiste el laboratorio. Pide información con `s.getRole("soloSecretosClase",{showPrivileges:true})` en lugar de borrarlo automáticamente.

Paso 4. Crear la cuenta de prueba

```
s.createUser({
  user:"auditorclase",
  pwd:process.env.LAB_READER_PASSWORD,
  roles:[{role:"soloSecretosClase",db:"curso_seguridad"}]
});
```

Para simplificar un entorno docente, esta identidad de demostración reutiliza el **valor ficticio local** `LAB_READER_PASSWORD` del contenedor. En producción se utilizarían credenciales independientes; nunca compartas una contraseña entre cuentas reales.

Si la cuenta ya existe, examina `s.getUser("auditorclase")`. No confundas un error por duplicidad de nombre con un error de permisos.

Paso 5. Probar una lectura permitida

Sal de `mongosh` con `exit`. Desde la terminal:

```
docker compose exec mongo-seguridad mongosh --username auditorclase --authenticationDatabase
curso_seguridad --password
```

Introduce la contraseña que asignaste. Dentro del shell:

```
db.getSiblingDB("curso_seguridad").secretos.findOne({_id:"demo-1"})
```

Resultado esperado: aparece el documento ficticio, porque el rol contiene la acción `find` sobre esta colección.

Paso 6. Probar una lectura que debe rechazarse

Sin cambiar de cuenta:

```
db.getSiblingDB("curso_seguridad").publicaciones_demo.findOne({_id:"publica-1"})
```

Resultado esperado: error de autorización. Hemos demostrado un permiso más preciso que el rol `read` aplicado a toda la base.

Si puedes consultar ambas colecciones, revisa qué roles reales tiene `auditorclase`. No declares el ejercicio aprobado hasta explicar por qué una cuenta con un supuesto rol limitado puede acceder a recursos adicionales.

Paso 7. Probar una inserción prohibida

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({
  _id:"no-autorizado-clase",
  texto:"prueba prohibida"
})
```

También debe rechazarse. Aunque el usuario pueda leer `secretos`, no tiene acción `insert`.

Paso 8. Documentar la matriz de permisos

Cuenta	Leer secretos	Leer publicaciones_demo	Escribir secretos
`auditorclase` con rol nuevo	Sí	No	No
`lectorlab` con rol read	Sí	Sí	No
`escritorlab` con rol readWrite	Sí	Sí	Sí

La segunda y tercera fila describen lo que esperamos de los roles anteriores en esta base. Si tu instalación no tiene las cuentas preparadas, realiza primero el laboratorio 1.

Paso 9. Preguntas de reflexión

1. ¿Qué diferencia hay entre asignar un rol `read` sobre la base completa y `find` sobre una única colección?
2. ¿Por qué el nombre del rol `soloSecretosClase` no basta para demostrar lo que permite?
3. ¿Qué riesgo aparecería si le otorgaras el rol `root` «para asegurarte de que funcione»?

4. ¿Qué prueba negativa necesitas para confirmar que el rol es realmente restrictivo?

Conclusión: el mínimo privilegio requiere concretar recursos y acciones, no simplemente crear usuarios con nombres que suenen seguros.

Laboratorio 6 — Cambiar permisos con control y comprobar que las sesiones reflejan la política

Propósito

A veces un permiso debe ampliarse temporalmente, por ejemplo para que un empleado pueda corregir datos durante una ventana autorizada. También debe poder retirarse después. Vamos a razonar sobre el ciclo de vida de un privilegio sin modificar la cuenta principal `lectorlab` del curso.

Aviso: esta actividad altera roles y usuarios de laboratorio. No la ejecutes en un sistema real ni sobre una cuenta ajena.

Paso 1. Crear una identidad temporal

En una sesión administrativa como en el laboratorio anterior:

```
const s = db.getSiblingDB("curso_seguridad");
s.createUser({
  user:"temporalclase",
  pwd:process.env.LAB_READER_PASSWORD,
  roles:[{role:"read",db:"curso_seguridad"}]
});
```

Su estado inicial es claro: puede consultar, pero no escribir. La contraseña de esta cuenta de demostración procede de `LAB_READER_PASSWORD`; no utilices ese patrón para cuentas reales.

Paso 2. Verificar antes de cambiar nada

Cierra la sesión administrativa y conecta como `temporalclase`:

```
docker compose exec mongo-seguridad mongosh --username temporalclase --authenticationDatabase
curso_seguridad --password
```

Comprueba lectura permitida:

```
db.getSiblingDB("curso_seguridad").secretos.findOne({_id:"demo-1"})
```

Después intenta:

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({_id:"temporal-1"})
```

Resultado esperado: denegación. Si este caso no se cumple, carecerá de sentido estudiar el cambio de permisos.

Paso 3. Autorizar temporalmente escritura

Sal de la sesión del usuario y vuelve a entrar como administrador. Dentro del shell administrativo:

```
const s = db.getSiblingDB("curso_seguridad");
s.grantRolesToUser(
  "temporalclase",
  [{role:"readWrite",db:"curso_seguridad"}]
)
```

Esta operación añade el rol `readWrite`. No es lo mismo que sustituir por completo la cuenta ni borrar el rol `read` anterior. Si vas a cambiar privilegios en una aplicación real, debes estudiar además cuándo las sesiones existentes perciben los cambios y cómo se gestionan sus credenciales.

Paso 4. Verificar el cambio con una conexión nueva

Cierra la conexión administrativa y abre una **nueva sesión** como `temporalclase` para evitar dudas de estado de autenticación:

```
docker compose exec mongo-seguridad mongosh --username temporalclase --authenticationDatabase
curso_seguridad --password
```

Ahora intenta insertar `{_id:"temporal-1",texto:"autorizado durante la ventana"}`.

Resultado esperado: operación permitida si el rol se asignó correctamente. Si existe ya un documento con ese identificador por una ejecución anterior, usa otro identificador o limpia únicamente tus datos de prueba, no toda la colección.

Paso 5. Retirar el privilegio temporal

Sal y vuelve como administrador:

```
const s = db.getSiblingDB("curso_seguridad");
s.revokeRolesFromUser(
  "temporalclase",
  [{role:"readWrite",db:"curso_seguridad"}]
);
```

La cuenta conserva su rol de lectura. No hemos bloqueado toda la identidad: hemos retirado una capacidad que dejó de ser necesaria.

Paso 6. Comprobar otra vez la restricción

Conecta de nuevo como `temporalclase` y prueba una inserción con un identificador nuevo:

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({_id:"temporal-prohibido"})
```

Resultado esperado: denegación. Comprueba también que la lectura sigue siendo posible.

La secuencia completa debería ser:

Estado	Leer	Insertar
Inicial	Sí	No
Permiso temporal concedido	Sí	Sí

Estado	Leer	Insertar
Permiso retirado	Sí	No

Paso 7. ¿Qué demuestra realmente el ejercicio?

No basta con que el administrador ejecute `grantRolesToUser` y `revokeRolesFromUser` sin errores. Debemos comprobar **desde la cuenta afectada** que las operaciones permitidas y denegadas corresponden con la política.

Pregunta de evaluación: ¿por qué usarías una cuenta temporal del laboratorio en lugar de ampliar directamente los poderes de `lectorlab` para todas las demás prácticas?

Laboratorio 7 — Leer el estado del servidor sin confundirlo con una auditoría de seguridad

El objetivo

Imagina que un compañero dice: «La conexión falla y no sé si es Docker, MongoDB o mis credenciales». Tu primera tarea será recopilar información de diagnóstico **sin exponer secretos** y sin cambiar permisos al azar.

Paso 1. Comprobar contenedores

Desde PowerShell o Bash en la raíz del proyecto:

```
docker compose ps
```

Anota qué servicios están encendidos. Un contenedor detenido no equivale a una contraseña equivocada: todavía no hemos alcanzado el servidor.

Paso 2. Revisar los últimos mensajes del servicio

```
docker compose logs --tail=80 mongo-seguridad
```

Busca errores de arranque, de inicialización o de configuración. No copies cientos de líneas sin interpretarlas. Anota la hora, el servicio y el fragmento que realmente ayude a comprender el problema.

Protección de información: no pegues claves privadas, contraseñas ni variables de entorno sensibles en tu informe.

Paso 3. Separar tres evidencias

- `docker compose ps` muestra el estado de los contenedores.
- `db.adminCommand({ping:1})` confirma que el servidor responde en una conexión adecuada.
- Una operación de lectura/escritura bajo una identidad concreta muestra si esa acción está autorizada.

Son tres evidencias diferentes. La primera no demuestra quién se ha autenticado; la segunda no concede privilegios; la tercera depende de la cuenta y del recurso consultado.

Paso 4. Repasar un rechazo real

Si `lectorlab` intenta insertar y recibe `Unauthorized`, debemos comprobar primero que sí consigue leer con esa misma sesión. Así aislamos el problema a los permisos de escritura, en vez de culpar a Docker o TLS.

Paso 5. No confundir registros técnicos con auditoría formal

Los logs de un contenedor muestran información operativa. Un sistema completo de auditoría de datos requeriría registrar hechos relevantes con garantías de integridad, retención y consulta. Ver una línea de arranque de `mongod` no demuestra por sí solo quién modificó un documento.

Ejercicio: redacta una respuesta de tres frases a alguien que muestra el log de arranque y afirma: «Ya hemos auditado todos los accesos». Explica qué falta comprobar.

Laboratorio 8 — Caso integrador: diseñar una política y demostrarla

El escenario

Una pequeña organización utiliza MongoDB para gestionar pedidos. Necesita:

1. una aplicación que consulte pedidos;
2. un servicio que registre pedidos nuevos;
3. una persona autorizada para administrar usuarios;
4. conexiones protegidas cuando la red lo exija;
5. una manera de demostrar que las cuentas no pueden realizar operaciones ajenas a su función.

Tu tarea es presentar una propuesta y **evidencias** en lugar de un conjunto de contraseñas y comandos copiados.

Paso 1. Dibujar los actores

Escribe los nombres de los tres actores. Para cada uno especifica:

- recursos que necesita consultar o modificar;
- operaciones imprescindibles;
- operaciones peligrosas que no deberían concederse;
- forma de autenticación adecuada para el ejemplo;
- entorno en el que se realiza la conexión.

La cuenta de administración no debería ser la identidad normal de la aplicación de consulta.

Paso 2. Elegir roles mínimos

Para la aplicación lectora, empieza estudiando `read` sobre la base de datos de pedidos. Para el servicio que registra operaciones, considera `readWrite` u otro rol más preciso si las operaciones realmente lo requieren. Para la administración, separa expresamente la responsabilidad de gestionar usuarios.

No diseñes una política basándote solo en «siempre usamos `root` porque es más fácil».

Paso 3. Proponer cuatro pruebas

Las pruebas deben incluir resultados tanto positivos como negativos:

Actor	Acción	Resultado esperado
Lector	Consultar pedido conocido	Permitido
Lector	Eliminar pedido	Denegado
Escritor	Insertar pedido válido	Permitido

Actor	Acción	Resultado esperado
Escritor	Gestionar roles administrativos	Denegado

Si solo realizas las dos pruebas positivas, aún no sabes si existe una separación efectiva de privilegios.

Paso 4. Relacionar con TLS

Explica por qué una conexión protegida con TLS puede ser necesaria aunque las cuentas tengan contraseñas. Después indica por qué TLS no basta para impedir que un usuario autenticado exceda sus permisos.

Paso 5. Producir un informe comprensible

Tu informe final debe describir el propósito de cada cuenta, sus permisos, los resultados observados en las pruebas y cualquier límite del ejercicio.

No está permitido escribir como conclusión «seguridad comprobada» si no has realizado las operaciones relevantes. Tampoco atribuyas a Community pruebas de LDAP o cifrado nativo que no hemos ejecutado.

Solución de referencia

Una solución coherente separa tres identidades y limita sus privilegios. La cuenta lectora puede consultar y no modificar; la cuenta de escritura puede registrar pedidos, pero no administrar usuarios; la cuenta administrativa queda reservada a tareas especiales. Las conexiones y los secretos se protegen de acuerdo con el contexto. Cada permiso importante se comprueba con una operación real, incluida una denegación esperada.

Criterio de evaluación: otra persona debe poder comprender qué protege tu propuesta, cuál es el límite de cada identidad y qué evidencia respalda tus afirmaciones.

Laboratorio 9 — Reconocer y proteger secretos de configuración

Objetivo y problema

Una aplicación que se conecta a MongoDB puede funcionar correctamente y al mismo tiempo exponer sus credenciales. Imagina que alguien sube a un repositorio público la contraseña administrativa dentro de un archivo de configuración. La aplicación seguirá conectando, pero una persona con acceso a ese fichero podría intentar utilizar la misma identidad.

El objetivo de esta práctica es distinguir **datos públicos de configuración**, **credenciales privadas** y **material criptográfico confidencial**. No intentaremos construir un gestor empresarial de secretos dentro de un curso introductorio.

Paso 1. Reconocer los archivos

Abre PowerShell o Bash en la raíz del repositorio. Debes localizar:

- `compose.yaml`: describe qué contenedores hay y qué parámetros necesitan.
- `.env.example`: muestra los nombres de variables esperados, con valores de demostración.
- `.env`: tu copia local, con las contraseñas que has elegido para la práctica.
- `.gitignore`: indica qué archivos locales no deben añadirse normalmente al control de versiones.

Que el archivo `.env` empiece por un punto no significa que esté cifrado. Es texto de configuración que debe custodiarse.

Paso 2. Crear la copia privada

En PowerShell:

```
Copy-Item .env.example .env
```

En Bash:

```
cp .env.example .env
```

Edita los valores de `MONGO_ADMIN_PASSWORD`, `LAB_READER_PASSWORD` y `LAB_WRITER_PASSWORD`. Utiliza secretos **ficticios y exclusivos de este laboratorio**, nunca contraseñas de correo, empresa u otros sistemas.

Paso 3. Clasificar cinco ejemplos

Dato	¿Publicarlo como ejemplo?	Razón
Nombre ficticio `lectorlab`	Sí	No demuestra identidad por sí solo

Dato	¿Publicarlo como ejemplo?	Razón
Puerto local `27018`	Sí	Describe la infraestructura
Contraseña verdadera	No	Podría permitir iniciar sesión
Certificado público de una CA de prueba	Según contexto	No es la clave privada
`server.pem` que incorpora clave privada	No	Contiene material secreto

El nombre de un usuario y la contraseña que demuestra ser ese usuario **no son lo mismo**.

Paso 4. Comprobar la exclusión del archivo

Si tienes Git instalado, ejecuta:

```
git check-ignore .env
```

El resultado correcto normalmente indica que `.env` está ignorado. Si no tienes Git, abre `.gitignore` en un editor y verifica la exclusión. **Advertencia:** ignorar hoy un fichero no borra una contraseña que ya haya sido publicada en un commit anterior.

Paso 5. Interpretar la sustitución de variables

El archivo Compose puede exigir una variable de contraseña administrativa. Al preparar los servicios, Docker Compose utiliza el valor de tu entorno o de `.env`. Eso evita escribir una contraseña concreta en el archivo compartido, pero **no cifra la variable ni sustituye un sistema profesional de secretos**.

No pegues la salida íntegra de comandos que muestran configuración resuelta si puede contener valores privados.

Paso 6. Resolver un fallo habitual

Un alumno inicia el servidor, luego cambia la contraseña en `.env` y deja de poder entrar.

Explicación: la cuenta administradora se creó en la base cuando se inicializó el volumen por primera vez. Editar después la variable local **no rota automáticamente la credencial almacenada en MongoDB**.

La solución es administrar y coordinar una rotación de contraseña. No elimines el volumen como respuesta instintiva; eso destruiría datos y cuentas de laboratorio.

Evaluación

Explica por qué `.env` no es un almacén cifrado, por qué `.gitignore` no revoca una clave previamente filtrada y por qué una contraseña no debería aparecer ni en el código ni en capturas del terminal.

Laboratorio 10 — Matriz de acceso: qué operaciones deben funcionar y cuáles no

Objetivo

Vamos a verificar que los usuarios de laboratorio no solo se autentican correctamente, sino que tienen permisos diferentes. Una cuenta lectora deberá consultar datos, pero no actualizarlos ni borrarlos. La cuenta escritora sí podrá modificar documentos, sin convertirse por ello en administradora de MongoDB.

Paso 1. Comprobar el servidor y las cuentas

En PowerShell o Bash, desde la raíz del repositorio:

```
docker compose up -d --wait mongo-seguridad
```

Las cuentas `lectorlab` y `escritorlab` deben existir desde el laboratorio 1. Si faltan, completa aquella preparación antes de continuar.

Paso 2. Conectar como lector

```
docker compose exec mongo-seguridad mongosh --username lectorlab --authenticationDatabase
curso_seguridad --password
```

Introduce la contraseña del lector. Dentro de `mongosh`:

```
db.getSiblingDB("curso_seguridad").secretos.findOne({_id:"demo-1"})
```

El documento debe aparecer. Esta es nuestra **prueba positiva**: demuestra que la identidad puede leer el recurso permitido.

Paso 3. Intentar actualizar un documento

```
db.getSiblingDB("curso_seguridad").secretos.updateOne(
  {_id:"demo-1"},
  {$set:{texto:"no autorizado"}}
)
```

Resultado esperado: denegación por falta de permiso de actualización.

Paso 4. Intentar borrar el documento

```
db.getSiblingDB("curso_seguridad").secretos.deleteOne({_id:"demo-1"})
```

También debe rechazarse. Comprueba después mediante `findOne` que el documento original continúa existiendo. La prueba no estaría bien si el mensaje pareciera correcto pero hubiéramos perdido datos.

Paso 5. Conectar como escritor

Sal de `mongosh` escribiendo `exit`. Desde la terminal:

```
docker compose exec mongo-seguridad mongosh --username escritorlab --authenticationDatabase
curso_seguridad --password
```

Dentro del shell, inserta un documento distinto:

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({
  _id:"matriz-escritor",
  texto:"insertado mediante rol readWrite"
})
```

Si ya existe ese identificador porque repetiste la actividad, cambia únicamente el identificador de tu documento ficticio; no confundas un error por duplicado con un error de autorización.

Paso 6. Actualizar y comprobar

```
db.getSiblingDB("curso_seguridad").secretos.updateOne(
  {_id:"matriz-escritor"},
  {$set:{texto:"actualizado por escritor"}}
)
db.getSiblingDB("curso_seguridad").secretos.findOne({_id:"matriz-escritor"})
```

Debe aparecer el texto actualizado. El rol `readWrite` permite escritura sobre esa base, pero **no concede por sí solo privilegios para administrar usuarios**.

Paso 7. Registrar resultados

Identidad	Consulta	Actualiza	Borra
`lectorlab`	Permitida	Denegada	Denegada
`escritorlab`	Permitida	Permitida	Permitida

Anota los mensajes reales y explica por qué dos cuentas autenticadas pueden recibir respuestas distintas al ejecutar la misma operación.

Conclusión correcta: los roles, no el simple hecho de introducir una contraseña válida, limitan qué acciones puede realizar cada identidad.